

Aula 07 · Expressões, Atribuição e Controle

Um resumo em linguagem de conversa, com os códigos da aula

Paradigmas de Linguagens de Programação · Engenharia de Software Base: Sebesta, *Conceitos de Linguagens de Programação*, 11ª ed., Caps. 7 e 8

Do que se trata esta aula?

A ideia central é simples: **seu código precisa fazer o que parece fazer**. Muita gente se atrapalha porque cada linguagem decide por conta própria como avaliar uma conta, converter um tipo ou escolher o caminho de um `if`. A aula mostra, com exemplos para rodar e prever, onde estão as pegadinhas, para você ler código dos outros sem ser enganado pela indentação, pela precedência ou por uma conversão silenciosa.

Três histórias reais mostram que isso não é detalhe acadêmico:

- **Apple, 2014 (“goto fail”)**. Uma linha `goto fail;` repetida por engano fazia o código pular a checagem da assinatura. Resultado: iPhones e Macs aceitavam certificados falsos.
- **Zune, 2008**. No último dia de um ano bissexto, um laço `while` nunca terminava e todos os aparelhos travaram.
- **PHP**. Comparar hashes com `==` fazia `"0e4 . . ."` ser igual a `"0e8 . . ."`, então senhas erradas eram aceitas.

Para esquentar: o que cada linha imprime?

Quiz de abertura · slide 4

A aula começa com seis palpites. Vale tentar responder antes de olhar a resposta embaixo de cada um.

1 · Python

main.py	
<pre>1 print(-2 ** 2, 2 ** 3 ** 2)</pre>	
OUTPUT	
<pre>-4 512</pre>	

→ Imprime **-4 512**. O `**` vem antes do menos unário e associa pela direita, então `2 ** (3 ** 2) = 512`.

2 · Java

Main.java	
<pre>1 System.out.println(1 + 2 + "3" + 4 + 5);</pre>	
OUTPUT	
<pre>3345</pre>	

→ Imprime **3345**. A soma vai da esquerda para a direita: `1+2` vira `3`; depois `"3"+"3"` vira texto `"33"`, e daí em diante só concatena.

3 · JavaScript

index.js	
<pre>1 console.log("5" - 2, "5" + 2);</pre>	
OUTPUT	
<pre>3 52</pre>	

→ Imprime **3 52**. O menos só faz sentido com números, então "5" vira 5. O mais prefere concatenar.

4 · C

```
main.c ▶ RUN
1 printf("%d\n", 3 > 2 > 1);
OUTPUT
0
```

→ Imprime **0**. Primeiro $3 > 2$ vale 1, e depois $1 > 1$ é falso.

5 · Python

```
main.py ▶ RUN
1 x = 0
2 print(x or "padrão", 3 > 2 > 1)
OUTPUT
padrão True
```

→ Imprime **padrão True**. O 0 conta como falso, e o `or` devolve o segundo operando. Já $3 > 2 > 1$ em Python é lido como $(3 > 2)$ e $(2 > 1)$.

6 · JavaScript

```
index.js ▶ RUN
1 console.log(0 == "", "" == "0", 0 == "0");
OUTPUT
true false true
```

→ Imprime **true false true**. O `==` converte os tipos antes de comparar, e por isso nem é transitivo.

1. Precedência e associatividade

Slides 5-6 · exemplos/01_precedencia

Antes de qualquer coisa, toda linguagem precisa responder: quem é calculado primeiro, e o que acontece quando há empate? Pegue $a + b * c$ com $a=3$, $b=4$ e $c=5$. Lendo da esquerda para a direita, daria 35. Respeitando a precedência (multiplicação antes), dá 23, que é o que quase todas as linguagens fazem. O APL, curiosamente, avalia sempre da direita para a esquerda, e o Lisp resolve tudo com parênteses e notação prefixa: $(+ a (* b c))$.

Na prática, as linguagens divergem em detalhes que pegam desprevenido:

- **Potência:** em Python $-2 ** 2$ dá **-4** (a potência vem antes do menos). Em JavaScript essa mesma linha é um erro de sintaxe, e o Java nem tem o operador `**`. No Excel, ao contrário, $= -2^2$ dá 4.
- **Potência em cadeia:** $2 ** 3 ** 2$ dá 512 em Python e JavaScript, porque a potência associa pela direita, ou seja, $2 ** (3 ** 2)$.
- **Somar número com texto:** $1 + 2 + "3"$ dá "33" em Java e JavaScript, mas Python se recusa e lança *TypeError*.
- **Divisão e resto com negativos:** $-7 / 2$ dá -4 em Python (com `//`), -3 em Java e -3.5 em JavaScript. Já $-7 \% 2$ dá 1 em Python e -1 em Java e JavaScript.

A regra de ouro da aula: **na dúvida, use parênteses**. Eles deixam sua intenção clara para o compilador e, principalmente, para a próxima pessoa que vai ler o código.

2. Ordem dos operandos e efeitos colaterais

Slide 7 · exemplos/02_efeitos_colaterais

Se uma função muda uma variável global enquanto a expressão ainda está sendo calculada, a ordem em que as partes são lidas muda o resultado. O exemplo do livro mostra isso: depois de `a = a + fun1()`, o `a` vale 8 ou 20?

C · exemplo do livro: 8 ou 20?

```
main.c
1  int a = 5;
2  int fun1(void) {
3      a = 17;          /* efeito colateral */
4      return 3;
5  }
6  int main(void) {
7      a = a + fun1();   /* 8 ou 20? */
8      printf("a = %d\n", a);
9  }
```

Depende de quem executa. O **Java dá 8**, porque a própria linguagem garante a leitura da esquerda para a direita. O **C dá 20**, porque o padrão deixa a ordem em aberto e cada compilador faz como quiser. O **Go também dá 20**: ele define a ordem das chamadas de função, mas não diz quando as variáveis são lidas em relação a elas.

Transparência referencial: quando as funções não têm efeitos colaterais, a ordem deixa de importar e $f(x) + f(x)$ pode até virar $2 * f(x)$. É por isso que funções puras são tão valorizadas.

3. Operadores sobrecarregados

Slide 8 · exemplos/03_sobrecarga

Sobrecarga é quando o mesmo símbolo faz coisas diferentes conforme os operandos. O `+` soma números, mas em Java também junta textos. Pode ser cômodo, mas também confunde: em C++, o `&` tanto é “E bit a bit” quanto “endereço de”. Algumas linguagens (C++, C#, Python, Rust, Kotlin) deixam você criar a sua própria sobrecarga; o Java não deixa.

Java

```
Main.java
1  System.out.println('a' + 1);          // 98
2  System.out.println("" + 'a' + 1);    // a1
3  x.add(y).multiply(tres);              // BigDecimal: sem +
```

Em Python, você ensina uma classe a lidar com `+` e `*` definindo métodos especiais:

Python · métodos especiais

```
main.py
1  class Vetor:
2      def __init__(self, x, y):
3          self.x, self.y = x, y
4      def __add__(self, outro):          # v1 + v2
5          return Vetor(self.x + outro.x,
6                        self.y + outro.y)
7      def __mul__(self, k):              # v * 3
8          return Vetor(self.x * k, self.y * k)
9  v = Vetor(1, 2) + Vetor(3, 4)         # Vetor(4, 6)
10 print(v * 3)                          # Vetor(12, 18)
```

4. Conversões de tipos

Slides 9-10 · exemplos/04_conversoes

Existem três situações que aparecem o tempo todo: o **alargamento** (um tipo menor cabe num maior, como int para float), o **estreitamento** (você perde informação, como double para int) e a **coerção** (a linguagem converte sozinha, sem avisar). Repare como o Java se comporta:

Java · coerção e cast

```
Main.java ▶ RUN
1  int grande = 123456789;
2  float f = grande;           // alargamento implícito
3  System.out.println((int) f); // 123456792 !
4  double d = 3.99;
5  System.out.println((int) d); // 3 (cast)
6  System.out.println((byte) 200); // -56
7  byte b = 10, c = 20;
8  // byte s = b + c; // erro: b + c é int (p. 312)
9  System.out.println(1 / 2);    // 0
10 System.out.println(1 / 2.0);  // 0.5
```

Cada linguagem tem uma filosofia. C e Java convertem int em real sozinhos. **Go e Rust se recusam**: você precisa escrever float64(i) ou i as f64, e misturar int32 com int64 é erro de compilação (como em ML e F#). Isso é chato de digitar, mas evita um problema sério: a coerção esconde erros. No exemplo do livro, se você digitar a no lugar de c em d = b * a;, o Java converte e compila sem reclamar.

Quem leva a coerção ao extremo são JavaScript e PHP:

JavaScript · coerção implícita

```
index.js ▶ RUN
1  "5" - 2           // 3
2  "5" + 2           // "52"
3  "5" * "2"         // 10
4  true + 1          // 2
5  [] + {}            // "[object Object]"
6  null + 1           // 1
7  undefined + 1      // NaN
8  "b" + "a" + +"a" + "a" // "baNaNaN"
9  0 == ""            // true
10 "" == "0"          // false
11 0 == "0"           // true (não é transitivo!)
12 0 === "0"          // false (=== não converte)
```

PHP · "type juggling"

```
index.php ▶ RUN
1  var_dump("0e123" == "0e456"); // true !
2  var_dump("0e123" === "0e456"); // false
3  var_dump("1" == "01"); // true
4  md5("240610708") // "0e462097431906..."
5  md5("QNKCDZO") // "0e830400451993..."
6  // == entre os dois hashes: true
```

Regra prática: em JavaScript e PHP, compare sempre com === e !==, que não fazem coerção. No PHP, o motivo do bug do início é que “0e123” e “0e456” parecem números em notação científica (os dois valem 0), e os hashes MD5 acima começam justamente assim.

5. Expressões booleanas e curto-circuito

Slides 11-12 · exemplos/05_booleanas

Em C, não existe um tipo booleano de verdade: verdadeiro é “qualquer coisa diferente de zero”.

Daí vem a pegadinha do quiz: a > b > c não significa o que parece.

C · booleano é inteiro

main.c

▶ RUN

```
1 int a = 3, b = 2, c = 1;
2 printf("%d\n", a > b > c); // 0 !
3 // (3 > 2) vale 1, e 1 > 1 é falso
4 printf("%d\n", (a > b) && (b > c)); // 1
5 if (-1) printf("verdadeiro\n"); // != 0
```

Python é mais gentil: ele aceita a comparação encadeada e entende `a > b > c` do jeito humano.

Mas atenção ao que o `if` considera falso ou verdadeiro:

Python · comparações encadeadas

main.py

▶ RUN

```
1 a, b, c = 3, 2, 1
2 print(a > b > c) # True: (a > b) and (b > c)
3 # contam como FALSO em um if:
4 # 0 0.0 "" [] {} None
5 # contam como VERDADEIRO:
6 # "0" [0] " "
```

C, Python e JavaScript aceitam qualquer valor dentro do `if`. Já Java, C#, Go e Rust exigem um booleano de verdade: `if (n)` com `n` inteiro nem compila, o que evita muita confusão.

O **curto-circuito** é quando o resultado já está decidido e a linguagem nem avalia o resto. Isso é muito útil para proteger acessos perigosos:

Java · && faz curto-circuito, & não

Main.java

▶ RUN

```
1 // busca do livro (p. 316)
2 while (i < lista.length && lista[i] != chave)
3     i++; // && evita lista[lista.length]
4 String s = null;
5 s != null && s.length() > 0 // false
6 s != null & s.length() > 0 // NullPointerException
```

JavaScript · ?. e ??

index.js

▶ RUN

```
1 const u = { nome: "Ana", endereco: null };
2 u.endereco?.cidade // undefined, sem erro
3 const qtd = 0;
4 qtd || 10 // 10 (o 0 se perdeu!)
5 qtd ?? 10 // 0 (só troca null/undefined)
```

Cuidado: em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. E em Python, `and` e `or` devolvem um dos operandos, não necessariamente `True` ou `False`: `0 or "padrão"` dá `"padrão"`.

6. Atribuição

Slides 13-14 · exemplos/06_atribuicao

Em C, atribuir também é uma expressão: ela devolve o valor atribuído. Dá para encadear (`sum = count = 0`) e usar dentro de condições, mas isso abre a porta para o erro clássico de escrever `=` querendo `==`.

C · a atribuição é uma expressão

```
main.c ▶ RUN
1  sum = count = 0;           // múltiplos alvos
2  a = b + (c = d / b) - 1;   // exemplo do livro
3  while ((ch = getchar()) != EOF) { ... }
4  if (x = y) { ... }        // compila! atribui e testa y
```

Cada linguagem lida com isso de um jeito. Em C, C++ e JavaScript, `if (x = y)` compila. Em Java e C# também é expressão, mas o `if` exige booleano e barra o erro. Python não permite, a não ser pelo operador “morsa” `:=`. Em Go, nem `x++` é expressão, e em Rust `if x = 5` simplesmente não compila.

Python · atribuição múltipla e operador morsa

```
main.py ▶ RUN
1  a, b = b, a                # troca sem temporária
2  if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Nas linguagens funcionais, `val` (ML) e `let` (F#, Rust) apenas dão um nome a um valor, que depois não muda.

Quando os tipos dos dois lados são diferentes, entra a **atribuição de modo misto**:

Java · só alargamento

```
Main.java ▶ RUN
1  double d = 3;              // ok: alargamento
2  // int i = 3.7;            // erro: estreitamento
3  int i = (int) 3.7;         // 3 (cast explícito)
4  long l = i;                // ok
5  i += 3.7;                  // COMPILA: cast escondido
6  System.out.println(i);     // 6
```

C · coerção em qualquer direção

```
main.c ▶ RUN
1  int i = 3.7;               // 3, em silêncio
2  char c = 300;              // 44 (só um aviso)
3  unsigned int u = -1;       // 4294967295
```

Armadilha clássica: `double taxa = acertos / total * 100;` com inteiros dá 0.0, porque a divisão inteira acontece antes da conversão para `double`.

7. Quantas estruturas de controle bastam?

Slide 15

Menos do que parece. Böhm e Jacopini provaram em 1966 que **sequência, seleção e laço** dão conta de qualquer algoritmo. O `goto` não é necessário. Então por que existem `for`, `switch`, `do` e `foreach`? Por conforto na escrita. O preço é a legibilidade: quanto maior a linguagem, menos do código alheio você reconhece de cara.

8. Seleção: o `else` pendente e o `switch`

Slides 16-17 · exemplos/07_selecao

Olhe este código C e responda: a quem pertence o `else`?

C · a endentação engana

```
main.c
1  int sum = 0, count = 5, result = -1;
2  if (sum == 0)
3      if (count == 0)
4          result = 0;
5  else // de qual if é este else?
6      result = 1;
7  printf("%d\n", result); // 1
```

Em Java, C, C++ e C#, o `else` sempre pertence ao `if` mais próximo que ainda não tem `else`, não importa como o texto está alinhado (só Python e F# levam a endentação a sério). Perl, Go e Rust resolvem exigindo chaves.

Em algumas linguagens, o próprio `if` devolve um valor:

Seletores que devolvem valor

```
exemplo.txt
1  let y = if x > 0 { x } else { 2 * x }; // Rust: if é uma expressão
2  y = x if x > 0 else 2 * x             # Python: expressão condicional
```

O `switch` de C tem uma pegadinha famosa: sem `break`, a execução “cai” para o caso seguinte.

C · sem break, o controle “cai”

```
main.c
1  switch (index) {
2      case 1:
3      case 3: odd += 1;
4      case 2:
5      case 4: even += 1;
6      default: printf("Error\n");
7  }
8  // index = 1: odd, even E erro!
```

As linguagens mais novas corrigiram isso. O Java 14 trouxe o `switch` com seta, que não cai e vale como expressão:

Java 14+ · switch como expressão

```
Main.java
1  String tipo = switch (dia) {
2      case SAB, DOM -> "fim de semana";
3      case SEG, TER, QUA,
4          QUI, SEX -> "dia útil";
5  }; // sem break, sem cair; exaustivo para enum
```

O Rust vai além com o `match`, que obriga você a cobrir todos os casos possíveis:

Rust · match exaustivo

```
main.rs
1  match n {
2      i32::MIN..=-1 => "negativo",
3      0 => "zero",
4      1..=9 => "um dígito",
5      _ => "vários dígitos",
6  }
7  // sem o _ : não compila
```

Em Go, cada caso termina sozinho e cair exige `fallthrough`. O C# proíbe cair sem querer, e o Python 3.10 ganhou o `match` com padrões e guardas.

9. Laços de repetição

Slides 18-19 · exemplos/08_iteracao

O `for` de C é, na verdade, um laço lógico disfarçado: você pode mexer na variável de controle dentro do corpo. Já no Python o `for` percorre uma sequência, e mexer no `i` não muda nada.

C · o `for` é um laço lógico disfarçado

main.c

▶ RUN

```
1  for (int i = 0; i < 10; i++) {
2      printf("%d ", i);
3      i += 2;          // altera a contagem
4  }                  // 0 3 6 9
5  for (; i <= 100; ) // expressões opcionais
6      soma += i++;
```

Python · o `for` percorre um iterável

main.py

▶ RUN

```
1  for i in range(0, 10):
2      print(i, end=" ")
3      i += 2          # NÃO muda nada
4                      # 0 1 2 3 4 5 6 7 8 9
5  for i in range(10, 0, -3): # 10 7 4 1
6      ...
```

Para sair de laços aninhados de uma vez, o Java oferece o `break` rotulado:

Java · `break` rotulado

Main.java

▶ RUN

```
1  externo:
2  for (int i = 0; i < m.length; i++)
3      for (int j = 0; j < m[i].length; j++)
4          if (m[i][j] < 0) {
5              System.out.println(i + ", " + j);
6              break externo; // sai dos DOIS laços
7          }
```

No JavaScript, uma letra faz toda a diferença: `for...in` percorre as *chaves*, e `for...of` percorre os *valores*.

JavaScript · `for...in` × `for...of`

index.js

▶ RUN

```
1  const notas = [10, 20, 30];
2  let total = 0;
3  for (const i in notas) total += i;
4  console.log(total); // "0012": chaves!
5  total = 0;
6  for (const v of notas) total += v;
7  console.log(total); // 60: valores
```

E aqui está o laço que derrubou os Zunes. Quando dias vale exatamente 366 num ano bissexto, nenhum ramo altera nada e o laço gira para sempre:

Zune (2008) · o laço sem progresso


```
main.c ▶ RUN
1 while (dias > 365) {
2     if (bissexto(ano)) {
3         if (dias > 366) { dias -= 366; ano++; }
4         // dias == 366: nada muda, laço infinito!
5     } else { dias -= 365; ano++; }
6 }
```

Uma lembrança: o `do-while` testa só no final, então executa pelo menos uma vez, e isso é fácil de esquecer.

10. Goto e comandos protegidos

Slides 20-21 · [exemplos/09_desvio_protegido](#)

O `goto` é a instrução mais poderosa e, justamente por isso, a mais perigosa. Veja uma versão simplificada do bug da Apple. O que `valida_certificado()` devolve?

"goto fail" (Apple, 2014) · versão simplificada

```
main.c ▶ RUN
1 int valida_certificado(void) {
2     int err;
3     if ((err = verifica_hash()) != 0)
4         goto fail;
5     goto fail; // duplicada: desvia SEMPRE
6     if ((err = verifica_assinatura()) != 0)
7         goto fail; // nunca é alcançada
8 fail:
9     return err; // 0 = "certificado válido"
10 }
```

Devolve 0, ou seja, “certificado válido”. O segundo `goto fail;` não pertence ao `if` (não há chaves, lembra do `else` pendente?), então sempre desvia e a verificação da assinatura nunca roda. Chaves obrigatórias (Go, Rust), o aviso do `gcc -Wall` e revisão de código teriam pego o erro.

Dijkstra publicou em 1968 o texto “Go To Statement Considered Harmful”. Hoje C, C++, C# e Go ainda têm `goto`, e o Java não. O que sobrou de bom são os `gotos` restritos: `break`, `continue` e `return`.

Dijkstra também propôs os **comandos protegidos**, em que cada ramo tem uma guarda. Todas as guardas são avaliadas; se várias forem verdadeiras, a escolha é **não determinística**; se nenhuma for, dá erro (no `if`) ou o laço termina (no `do ... od`). A ideia inspirou o CSP de Hoare e o `select` do Go.

Comandos protegidos de Dijkstra (exemplo do livro)

exemplo.txt

 RUN

```
1  if x >= y -> max := x
2  [] y >= x -> max := y
3  fi
```

Go · select, herdeiro dos comandos protegidos

main.go

 RUN

```
1  a <- "a"      // os dois canais
2  b <- "b"      // estão prontos
3  select {      // Go sorteia uma guarda
4  case x := <-a:
5      contagem[x]++
6  case x := <-b:
7      contagem[x]++
8  }
9  // 1000 repetições:
10 // map[a:505 b:495] (muda a cada execução)
```

11. Exercício: encontre o bug

Slides 22-24 · exemplos/10_exercicio

Para cada trecho, pergunte: qual é a saída, qual regra da linguagem causa o problema e como corrigir. Tente antes de ler a resposta.

1 · JavaScript

index.js

 RUN

```
1  const precos = [10, 20, 30];
2  let total = 0;
3  for (const p in precos)
4      total += p;
5  console.log("Total: " + total);
```

OUTPUT

Total: 0012

→ Imprime "Total: 0012". O `for...in` percorre as chaves (que são textos) e o `+` concatena.

Troque por `for...of` ou use `precos.reduce((a, b) => a + b, 0)`.

2 · Python

main.py

 RUN

```
1  def desconto(preco, d=None):
2      d = d or 10      # padrão: 10%
3      return preco * (100 - d) / 100
4  print(desconto(200, 0))
```

OUTPUT

180.0

→ Imprime 180.0, quando o certo seria 200.0. O `d or 10` trata o 0 como falso. Escreva `if d is None: d = 10` (em JavaScript, o `??` resolveria).

3 · Java

Main.java RUN

```
1 int dia = 2; String nome = "";
2 switch (dia) {
3     case 1: nome = "domingo";
4     case 2: nome = "segunda";
5     case 3: nome = "terça";
6     default: nome = "inválido";
7 }
8 System.out.println(nome);
```

OUTPUT

inválido

→ Imprime “inválido”. Sem `break`, o `switch` cai do caso 2 até o `default`. Coloque `break` em cada caso ou use o `switch` com seta (Java 14+).

4 · C

main.c RUN

```
1 int saldo = 100, saque = 50;
2 if (saque > saldo);
3     printf("saldo insuficiente\n");
4     printf("fim\n");
```

OUTPUT

saldo insuficiente
fim

→ Imprime “saldo insuficiente” e “fim”, mesmo com saque menor que o saldo. O ponto e vírgula depois do `if` é uma instrução vazia, e o `printf` fica fora. Use sempre chaves (o `gcc -Wall` avisa).

5 · Go

main.go RUN

```
1 x := 10
2 if x > 5 {
3     x := x * 2
4     fmt.Println("dentro:", x)
5 }
6 fmt.Println("fora:", x)
```

OUTPUT

dentro: 20
fora: 10

→ Imprime “dentro: 20” e “fora: 10”. O `:=` dentro do `if` cria uma *nova* variável `x` (sombreamento) em vez de alterar a de fora. Use `x = x * 2`.

6 · Java

Main.java RUN

```
1 int acertos = 7, total = 10;
2 double taxa = acertos / total * 100;
3 System.out.println(taxa + "%");
```

OUTPUT

0.0%

→ Imprime "0.0%". A divisão inteira $7 / 10$ dá 0 *antes* de virar double. Escreva `acertos * 100.0 / total` ou `(double) acertos / total * 100`.